

Tutors Monorepo Architecture

Comprehensive Architectural Overview

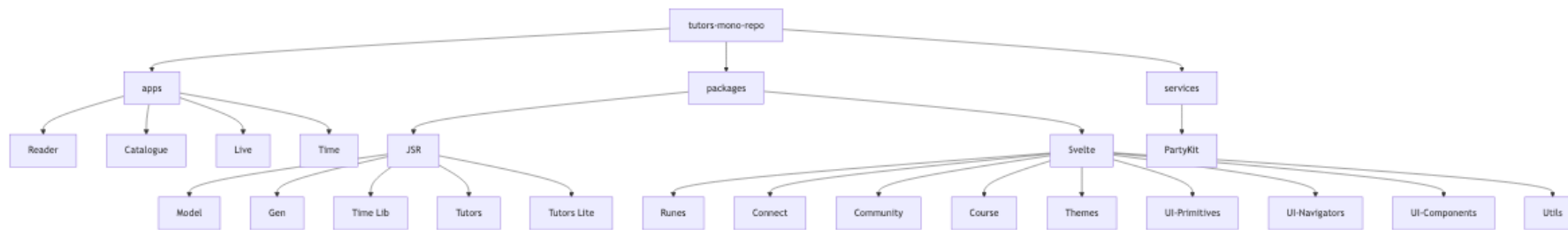
Overview

The **Tutors Monorepo** unifies the complete Tutors educational platform:

- **Course Generation** — Markdown to structured course data (JSON or static HTML)
- **Course Reading** — Interactive web-based viewer
- **Platform Features** — i18n, accessibility, OAuth, live collaboration
- **Time Analytics** — Student engagement tracking and visualisation

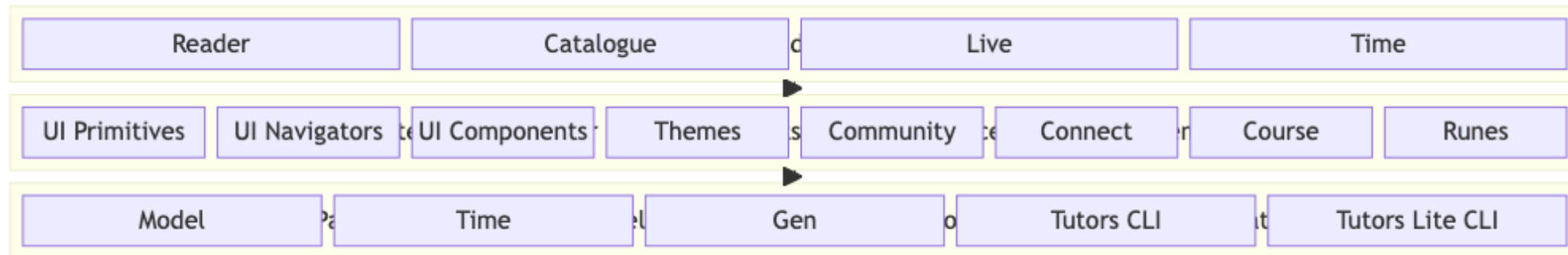
4 Subsystems: JSR Packages, Svelte Packages, Applications, Services

Repository Structure



Architectural Layers

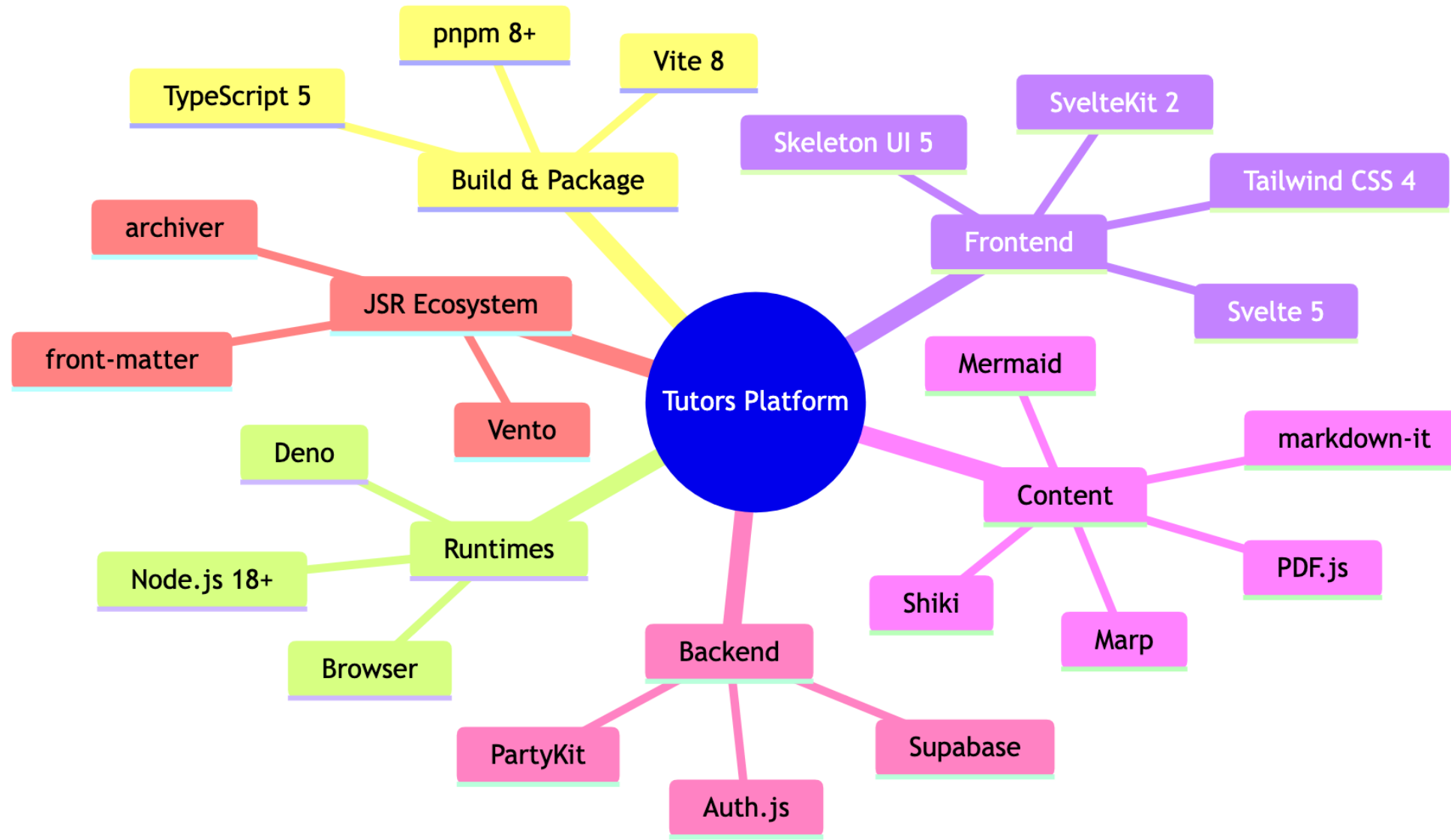
Three layers with strict bottom-up dependencies:



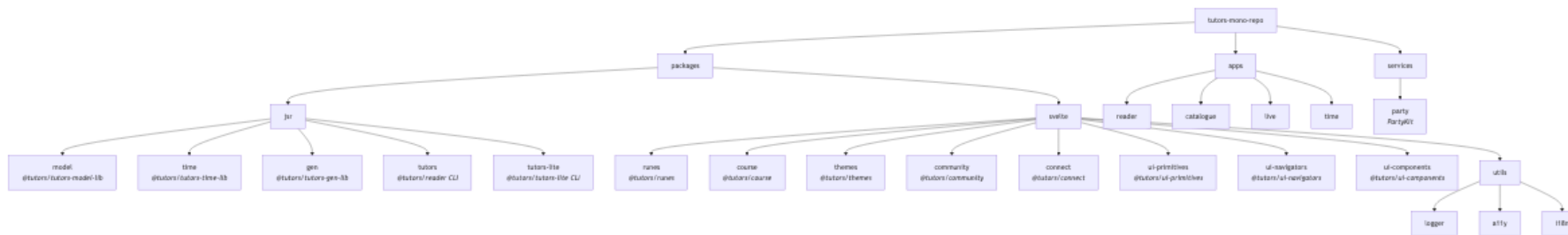
Key Principles

1. **Bottom-Up Dependencies** — Lower layers never depend on higher layers
2. **Package Isolation** — Clear boundaries and responsibilities
3. **Shared Types** — JSR packages provide TypeScript types across the ecosystem
4. **Runtime Compatibility** — JSR packages work in both Deno and Node.js
5. **Svelte-Specific** — Svelte packages leverage Svelte 5 reactivity

Technology Stack



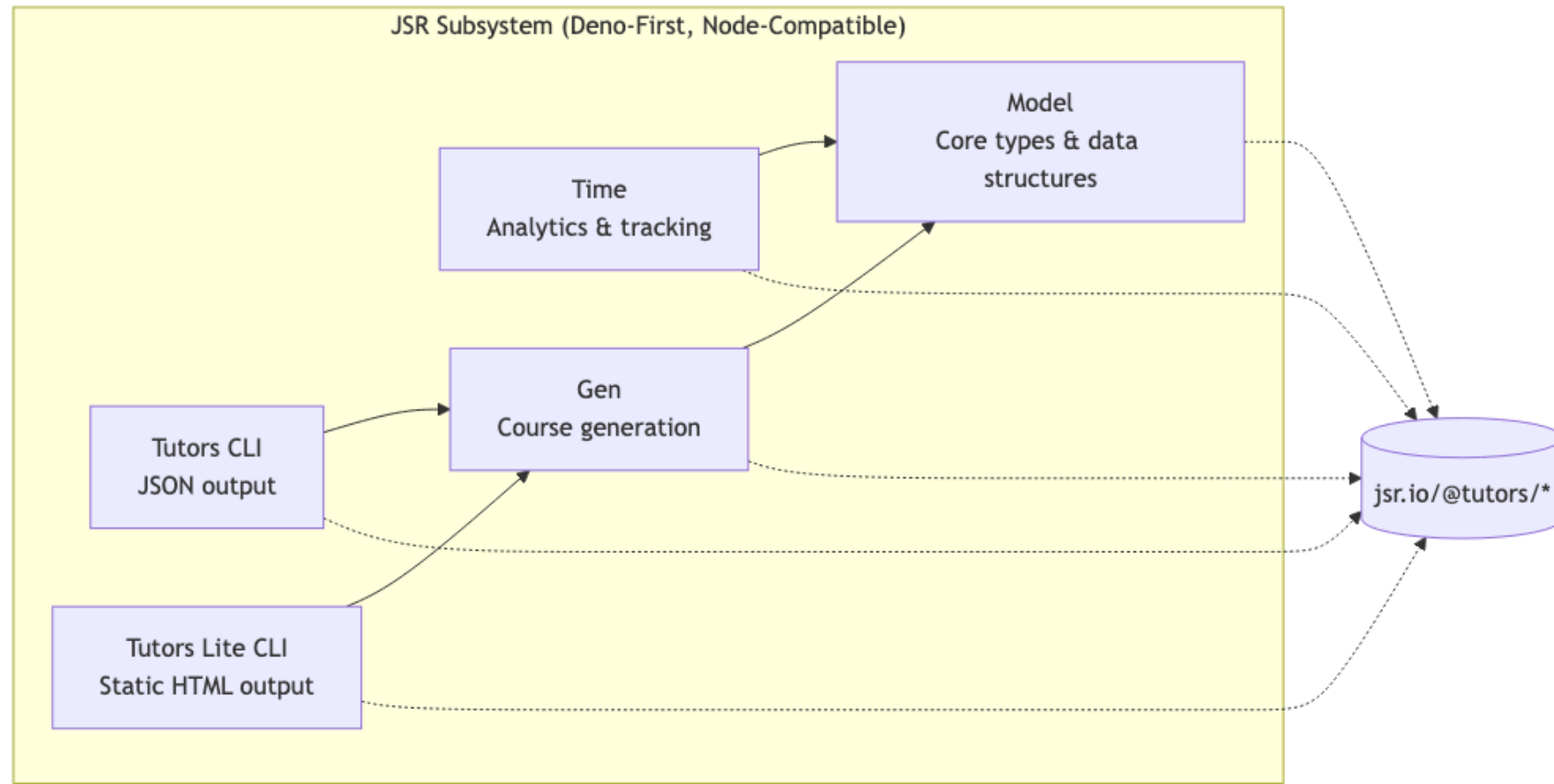
Package Structure — Directory Layout



JSR Subsystem

Foundation layer — core data models, course generation, CLI utilities.

Deno-first, Node.js compatible. Published to `jsr.io/@tutors/*`.



JSR — Model Package

@tutors/tutors-model-lib — Core data structures & types

```
interface Lo {  
    type: LoType;           // "course" | "topic" | "lab" | "talk" | etc.  
    title: string;  
    route: string;  
    los?: Lo[];             // Child learning objects  
}  
  
interface Course extends Lo {  
    type: "course";  
    url: string;  
    topicIndex: Map<string, Lo>;  
}
```

JSR — Gen Package

`@tutors/tutors-gen-lib` — Course content generation

- **CourseBuilder** — Scan folders, identify Lo types, build course tree
- **ResourceBuilder** — Parse markdown frontmatter & content
- **TemplateEngine** — Vento-based HTML rendering
- **CourseEmitter** — Write `tutors.json` or static HTML

JSR — Tutors Lite Package

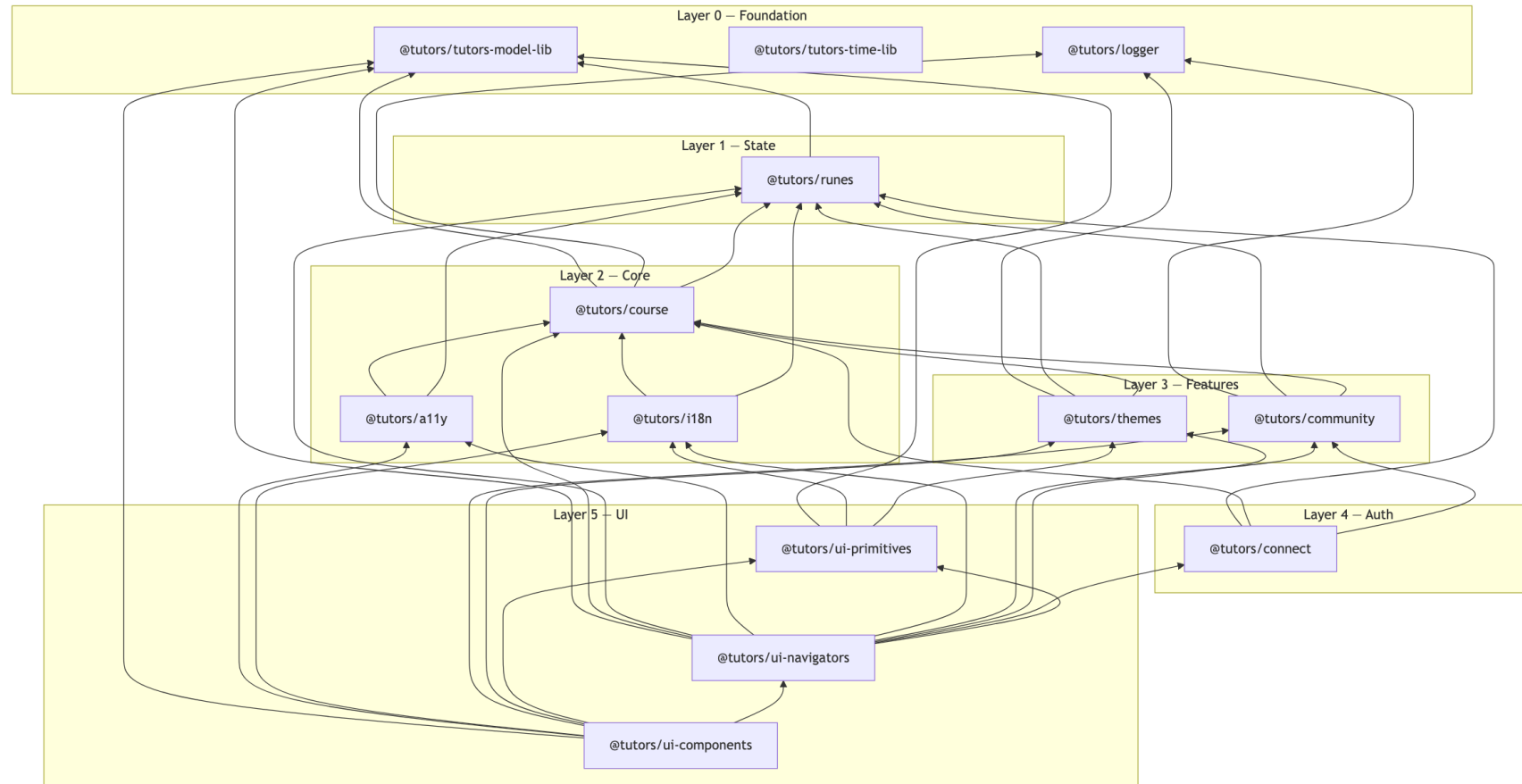
`@tutors/tutors-lite` — Static HTML course generation

Aspect	Tutors	Tutors Lite
Output	<code>json/tutors.json</code>	<code>html/</code> static site
Gen function	<code>generateDynamicCourse()</code>	<code>generateStaticCourse()</code>
Templates	None (JSON only)	Vento (<code>.vto</code>)
Viewer	Reader web app	Self-contained, open in browser

```
deno run -A jsr:@tutors/tutors-lite
```

Svelte Package Ecosystem

Six layers from foundation to UI — each building on the layer below.



Runes Package

`@tutors/runes` — Global reactive state using Svelte 5 runes

```
export const currentCourse = {  
  get value() {  
    return (_currentCourse ??= rune(null)).value;  
  },  
  set value(v) {  
    (_currentCourse ??= rune(null)).value = v;  
  }  
};
```

Lazy initialization avoids SSR issues.

Course Package

`@tutors/course` — Course loading, processing, markdown rendering

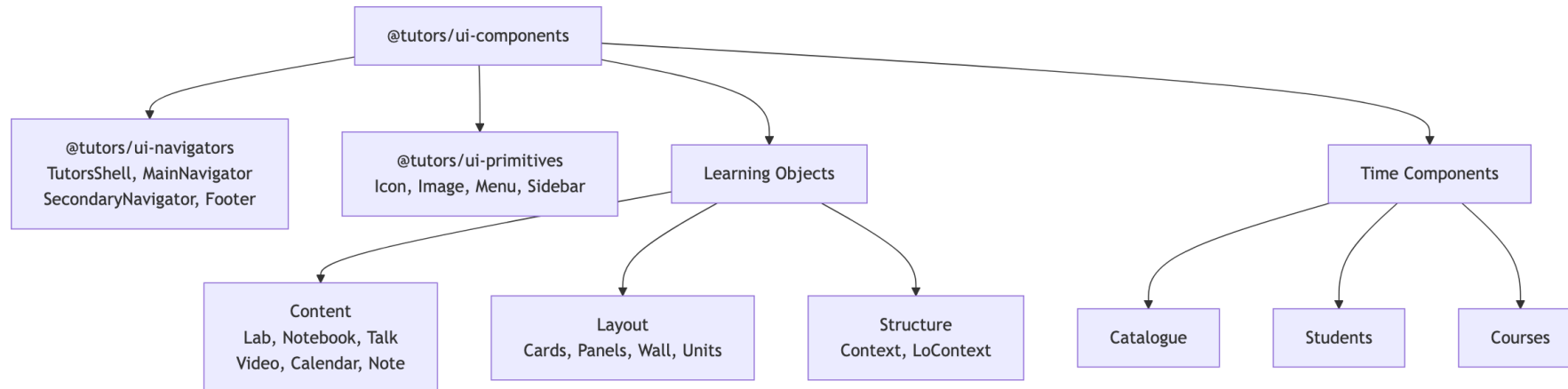
- **courseService** — Fetch JSON, cache, decorate tree
- **LiveLab** — Lab step navigation & markdown fetching
- **markdownService** — Shiki highlighting, Mermaid diagrams, Marp rendering

UI Package Trio

@tutors/ui-primitives — Icon, Image, Menu, Sidebar, LoContextTree, StudentCard

@tutors/ui-navigators — TutorsShell, MainNavigator, SecondaryNavigator, Footer

@tutors/ui-components — Learning objects, time views + pre-compiled CSS



Applications — SvelteKit Architecture

All apps follow the same structure:

- **Reader** — Main course viewer (/course , /lab , /talk , /video , /note)
- **Catalogue** — Course discovery & statistics
- **Live** — Real-time student presence tracking
- **Time** — Student engagement analytics (AG Grid, calendar heatmaps, Supabase)

Route groups: (auth)/ , (course-reader)/ , (home)/

Supabase Realtime — Presence & Live Activity

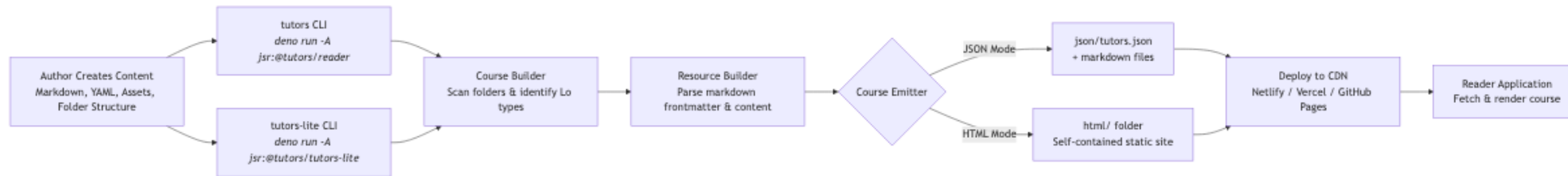
Channel-based broadcast — one channel per course, one global channel for platform-wide activity.

All presence features use **Supabase Realtime Broadcast channels**:

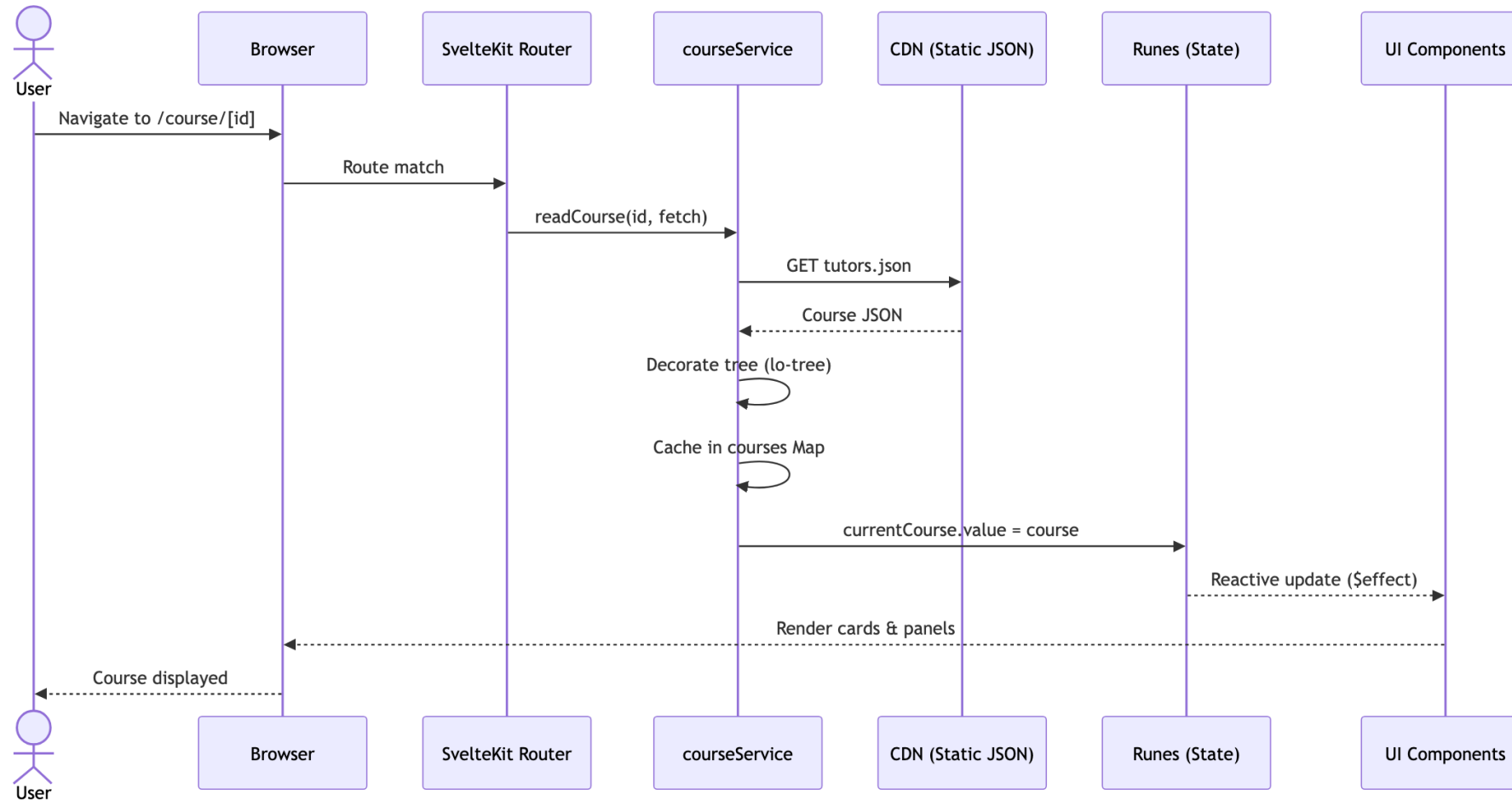
- `tutors-all-course-access` — global channel for platform-wide student tracking
- Per-course channels — keyed by `courseId`, track students in a specific course
- `lo-event` broadcast type carries `LoRecord` payloads (course, LO, user info)

PartyKit is retained only for **whiteboard collaboration** (Excalidraw + Durable Object storage).

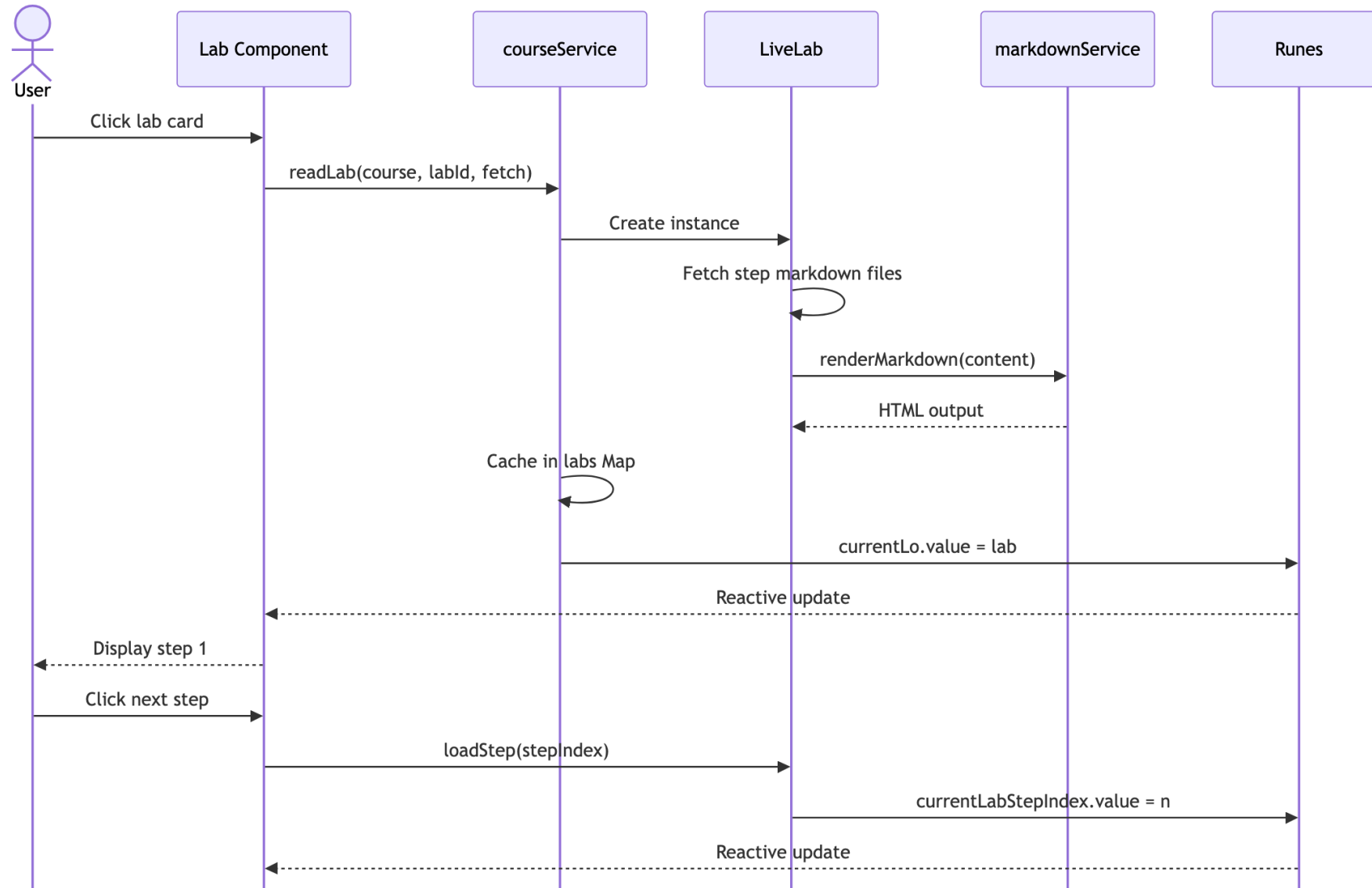
Content Generation Pipeline



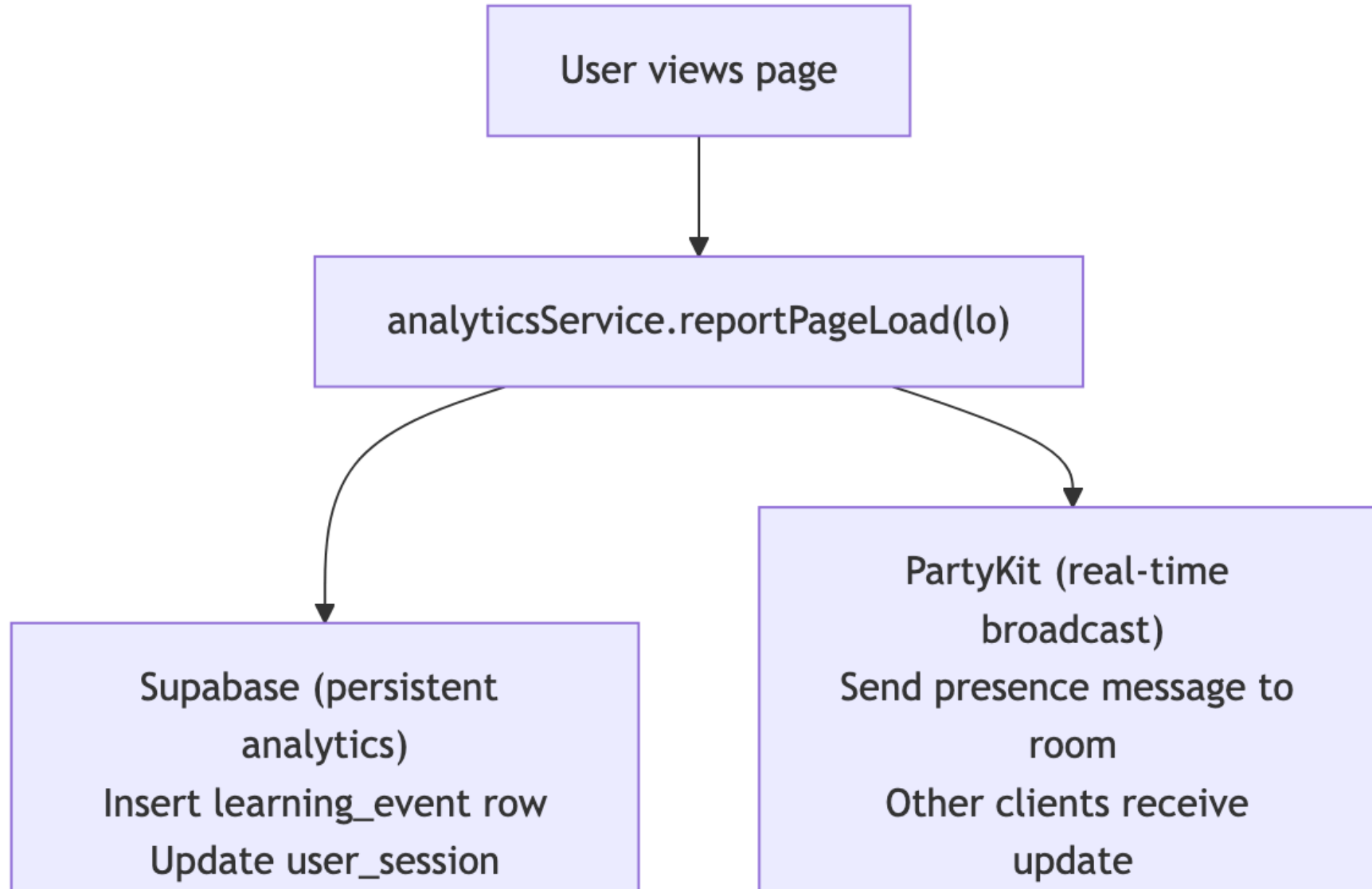
Reader — Course Loading Sequence



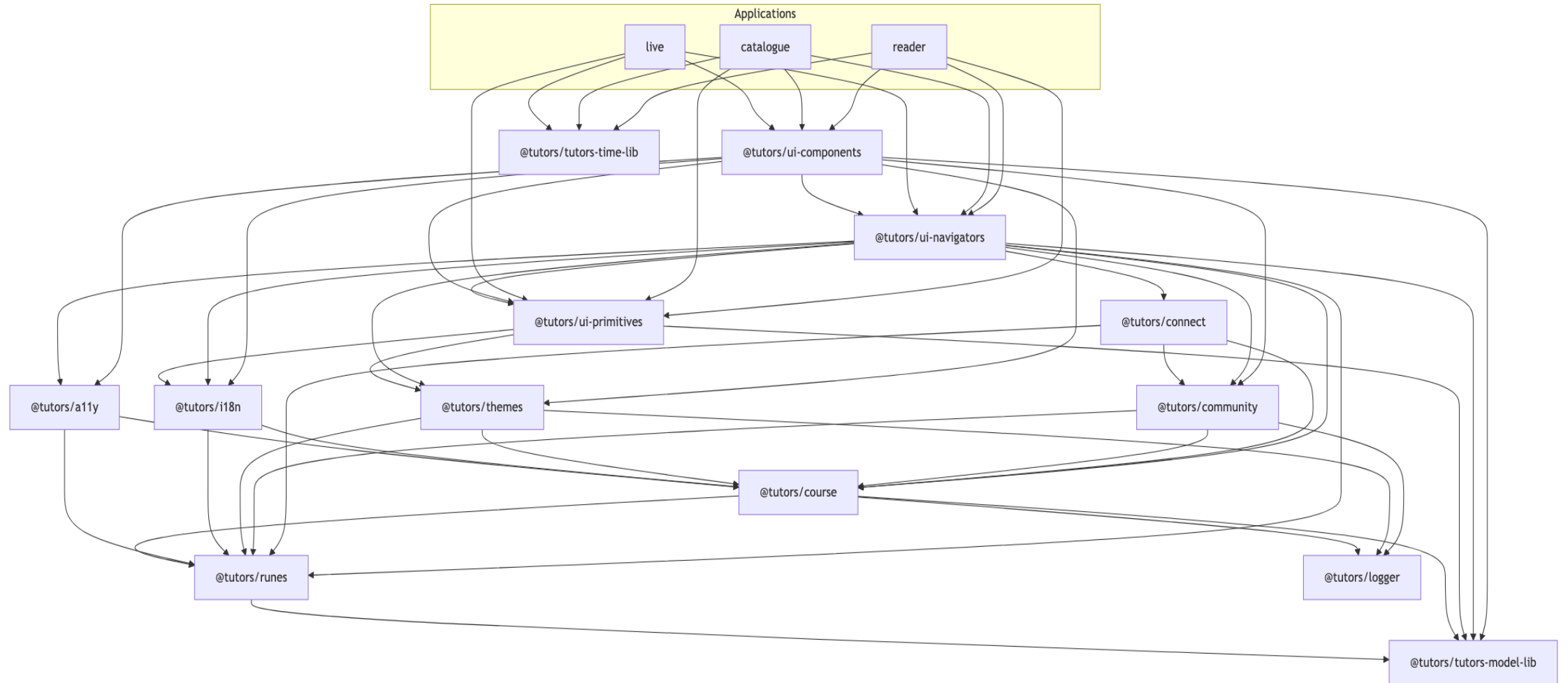
Reader — Lab Viewing Sequence



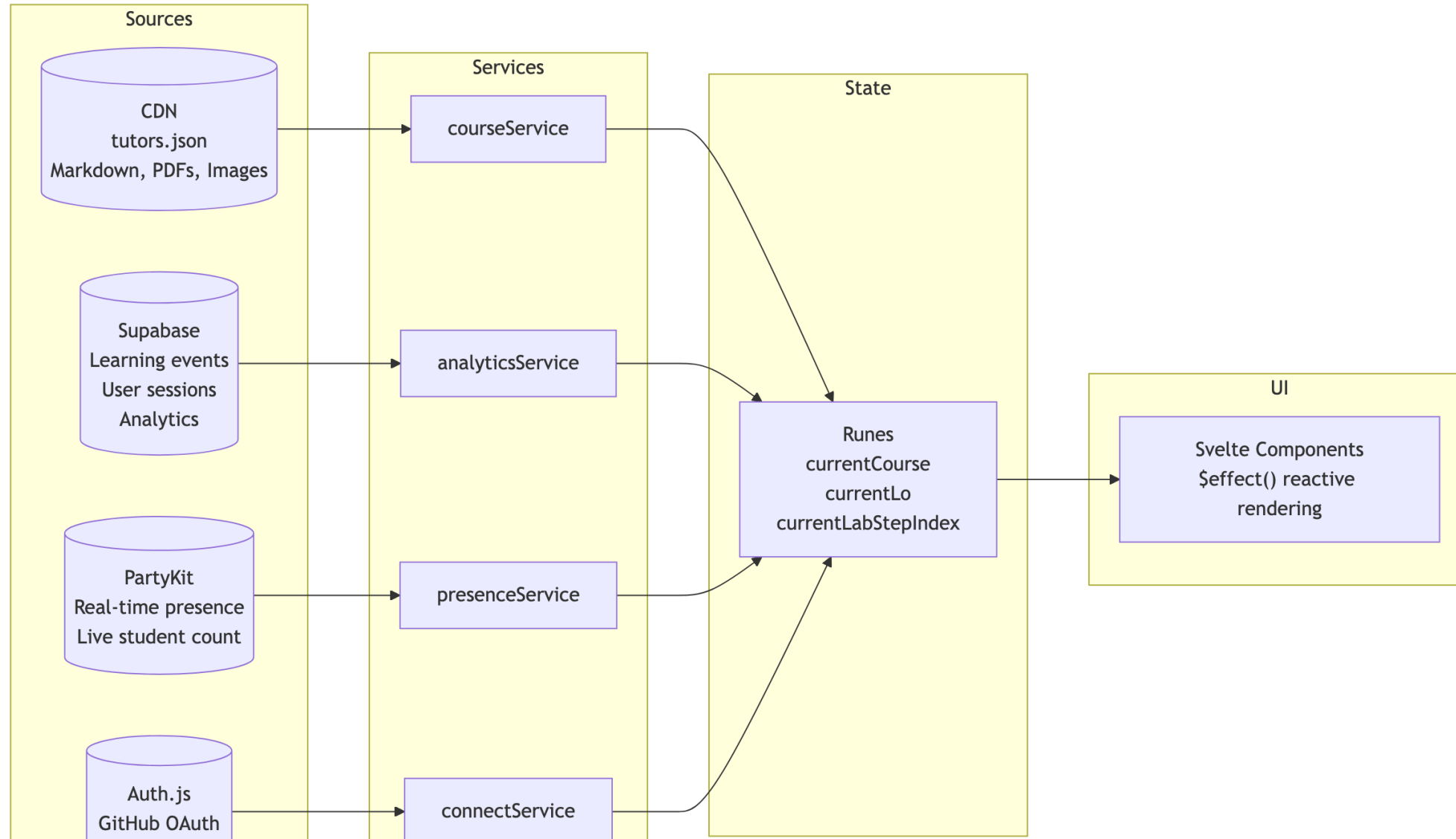
Analytics & Presence Flow



Dependency Graph



Data Flow & State Management



Development Workflow

```
# Install & run
pnpm install
cd apps/reader && pnpm dev      # http://localhost:5173

# Anonymous mode (no backend)
echo "PUBLIC_ANON_MODE=TRUE" > apps/reader/.env

# Build UI packages (required before app build)
pnpm --filter @tutors/ui-primitives build
pnpm --filter @tutors/ui-navigators build
pnpm --filter @tutors/ui-components build
pnpm --filter tutors-reader build
```

Deployment

- **JSR Packages** → `deno publish` to jsr.io
- **Reader App** → `pnpm build` → Netlify / Vercel (auto-detected adapter)
- **Presence** → Supabase Realtime (no separate deployment — part of Supabase project)
- **Whiteboard** → `npx partykit deploy` (PartyKit, for Excalidraw collaboration only)
- **Courses (JSON)** → `deno run -A jsr:@tutors/reader` → deploy `json/` to any static host
- **Courses (HTML)** → `deno run -A jsr:@tutors/tutors-lite` → deploy `html/` to any static host

Contributing

1. Fork → Clone → Branch from `development`
2. Follow commit convention: `Add:` , `Update:` , `Fix:` , `Refactor:`
3. Run `pnpm -r check` for type safety
4. Submit PR to `development`

Branch Strategy: `main` (production) ← `development` ← `feature/*` / `fix/*`